

Lecture 1: GitHub Actions for ML Pipelines

Train → Evaluate → Save Model → Simulate Registry

MLOps Foundations

1. Lecture Goal

In this lecture, we move from general CI ideas into an **MLOps use case** using **GitHub Actions**. Instead of compiling and testing ordinary software, our pipeline will:

1. train a machine learning model,
2. evaluate its performance,
3. save the trained model and metrics,
4. simulate a simple model registry,
5. upload artifacts to GitHub Actions.

We are **not** using a hosted MLflow server in this lecture. Instead, we combine two practical ideas:

- use **GitHub Actions Artifacts** to save outputs,
- use a local file-based **mock registry** to simulate model registration.

2. Why This Lecture Matters

In traditional software CI, a push to GitHub may trigger:

- installation of dependencies,
- code linting,
- unit tests,
- packaging.

In ML systems, this is not enough. We often need:

- model training,
- metric computation,
- model artifact generation,
- performance comparison,
- selection of a candidate or champion model.

So this lecture demonstrates how **GitHub Actions can automate an ML pipeline**.

3. Learning Outcomes

By the end of this lecture, students should be able to:

1. explain how GitHub Actions fits into an ML pipeline,
2. create a workflow file under `.github/workflows/`,
3. run model training automatically on a Git push,
4. save model files and metrics as artifacts,
5. simulate model registration using files and folders.

4. The Running Example

We will use a small `scikit-learn` classification example so that the pipeline is simple and reproducible.

Our pipeline logic will be:

```
push to GitHub → install dependencies → run training script → save metrics/model  
→ update mock registry → upload artifacts
```

5. Project Structure

We will create the following project:

```
mlops-github-actions-demo/  
  
  .github/  
    workflows/  
      ml_pipeline.yml  
  
  src/  
    train.py  
  
  outputs/  
    metrics.json  
    model.pkl  
  
  registry/  
    champion.json  
  
  requirements.txt  
  README.md
```

6. Step 1: Create the Project Folder

Create a project directory and move into it:

```
mkdir mlops-github-actions-demo  
cd mlops-github-actions-demo
```

Create the needed folders:

```
mkdir -p .github/workflows
mkdir -p src
mkdir -p outputs
mkdir -p registry
```

7. Step 2: Initialize Git

Initialize a Git repository:

```
git init
```

Create a GitHub repository through the web interface, name it mlops-github-actions-demo, then connect it:

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
```

8. Step 3: Create the Requirements File

Create requirements.txt:

```
scikit-learn
pandas
joblib
```

9. Step 4: Write the Training Script

Create src/train.py with the following code.

```
import json
import os
from pathlib import Path

import joblib
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split

# Create output folders if they do not already exist
Path("outputs").mkdir(parents=True, exist_ok=True)
Path("registry").mkdir(parents=True, exist_ok=True)

# 1. Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Train model
model = RandomForestClassifier(
    n_estimators=10,
    random_state=42
)
model.fit(X_train, y_train)

# 4. Evaluate
preds = model.predict(X_test)
accuracy = accuracy_score(y_test, preds)
f1 = f1_score(y_test, preds)

metrics = {
    "accuracy": round(accuracy, 4),
    "f1_score": round(f1, 4)
}

# 5. Save model
joblib.dump(model, "outputs/model.pkl")

# 6. Save metrics
with open("outputs/metrics.json", "w") as f:
    json.dump(metrics, f, indent=4)

# 7. Simulate a simple registry
registry_path = "registry/champion.json"

new_candidate = {
    "model_name": "Random Forest Classifier",
    "model_path": "outputs/model.pkl",
    "metrics_path": "outputs/metrics.json",
    "accuracy": metrics["accuracy"],
    "f1_score": metrics["f1_score"],
    "status": "candidate"
}

print(new_candidate)

if os.path.exists(registry_path):
    with open(registry_path, "r") as f:
        current_champion = json.load(f)

    # Replace champion only if the new model has better F1
    if new_candidate["f1_score"] > current_champion.get("f1_score", 0):
        new_candidate["status"] = "champion"
        with open(registry_path, "w") as f:
            json.dump(new_candidate, f, indent=4)
        print("New model promoted to champion.")
    else:
        print("Current champion retained.")
else:
```

```
new_candidate["status"] = "champion"
with open(registry_path, "w") as f:
    json.dump(new_candidate, f, indent=4)
print("No champion existed. New model set as champion.")

print("Training complete.")
print("Metrics:", metrics)
```

10. Step 5: Understand What the Script Does

This script performs five ML pipeline stages:

1. **Load data** from a built-in dataset.
2. **Train** a random forest classifier.
3. **Evaluate** the model using accuracy and F1.
4. **Save artifacts** such as `model.pkl` and `metrics.json`.
5. **Simulate model registration** using `registry/champion.json`.

This is not a real hosted registry, but it teaches the same idea: **we keep track of which model is currently considered the best one.**

11. Step 6: Create the GitHub Actions Workflow

Now create the workflow file:

```
touch .github/workflows/ml_pipeline.yml
```

Paste the following YAML into `.github/workflows/ml_pipeline.yml`:

```
name: ML Pipeline with GitHub Actions

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  train-evaluate-register:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
```

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt

- name: Run training pipeline
  run: |
    python src/train.py

- name: Show generated files
  run: |
    ls -R
    echo "----- metrics.json -----"
    cat outputs/metrics.json
    echo "----- champion.json -----"
    cat registry/champion.json

- name: Upload model artifact
  uses: actions/upload-artifact@v4
  with:
    name: trained-model
    path: outputs/model.pkl

- name: Upload metrics artifact
  uses: actions/upload-artifact@v4
  with:
    name: model-metrics
    path: outputs/metrics.json

- name: Upload registry artifact
  uses: actions/upload-artifact@v4
  with:
    name: model-registry
    path: registry/champion.json
```

12. Step 7: Understand the Workflow File

Let us interpret the workflow step by step.

12.1. Workflow Name

```
name: ML Pipeline with GitHub Actions
```

This is just the display name shown in the GitHub Actions tab.

12.2. Triggers

```
on:
  push:
    branches:
      - main
```

```
workflow_dispatch:
```

This means the workflow runs when:

- code is pushed to the `main` branch,
- or a user manually runs it from GitHub.

12.3. Job Definition

```
jobs:  
  train-evaluate-register:  
    runs-on: ubuntu-latest
```

A **job** is a major unit of work. Here we define one job that runs on a GitHub-hosted Ubuntu machine.

12.4. Checkout the Repository

```
- name: Checkout repository  
  uses: actions/checkout@v4
```

This downloads your repository files into the runner so the workflow can access them.

12.5. Set Up Python

```
- name: Set up Python  
  uses: actions/setup-python@v5  
  with:  
    python-version: "3.11"
```

This installs Python on the runner.

12.6. Install Dependencies

```
- name: Install dependencies  
  run: |  
    python -m pip install --upgrade pip  
    pip install -r requirements.txt
```

This installs the libraries your ML code needs.

12.7. Run the ML Pipeline

```
- name: Run training pipeline  
  run: |  
    python src/train.py
```

This is the core MLOps step. GitHub Actions is no longer just testing code; it is running an ML training script.

12.8. Upload Artifacts

```
- name: Upload model artifact
  uses: actions/upload-artifact@v4
  with:
    name: trained-model
    path: outputs/model.pkl
```

This stores the trained model in GitHub Actions so it can be downloaded later from the workflow run.

We do the same for metrics and registry files.

13. Step 8: Add a README

Create a simple README.md:

```
# MLOps GitHub Actions Demo

This project demonstrates a basic ML pipeline in GitHub Actions:

- train a model
- evaluate it
- save model and metrics
- simulate model registration
- upload artifacts

Run locally:
python src/train.py
```

14. Step 9: Commit and Push

Now commit everything:

```
git add .
git commit -m "Add GitHub Actions ML pipeline demo"
git branch -M main
git push -u origin main
```

After pushing, go to the **Actions** tab of your GitHub repository. You should see the workflow start automatically.

15. Step 10: What you Should Observe in GitHub

You should inspect the following:

1. The workflow run status.
2. The logs for each step.
3. The generated `metrics.json`.
4. The uploaded artifacts.
5. The `champion.json` file that acts as a mock registry.

16. Example Output Files

A sample `outputs/metrics.json` might look like:

```
{
  "accuracy": 0.9561,
  "f1_score": 0.9655
}
```

A sample `registry/champion.json` might look like:

```
{
  "model_name": "random_forest_classifier",
  "model_path": "outputs/model.pkl",
  "metrics_path": "outputs/metrics.json",
  "accuracy": 0.9561,
  "f1_score": 0.9655,
  "status": "champion"
}
```

17. Classroom Walkthrough Plan

A good live lecture flow would be:

1. Show the project structure.
2. Write `train.py` first.
3. Run it locally.
4. Create the workflow YAML.
5. Push to GitHub.
6. Open the Actions tab.
7. Inspect logs and artifacts.

18. Local Test Before Pushing

Before relying on GitHub Actions, test the pipeline locally:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python src/train.py
```

Then inspect:

```
cat outputs/metrics.json
cat registry/champion.json
```